

Problem Solving Using C (25VBT0C103)



www.onlinejain.com

Module: 02

Functions, Arrays & Strings

Module Information

Field	Details
Course Title	Problem Solving using C
Course Code	25VBT0C103
Module Number	Module 2 of 5
Module Title	Functions, Arrays and Strings
Semester	I
Credits	4

Module Overview

In Module 1, you learned to write C programs that make decisions and repeat tasks. But every real-world software system — whether it is the UPI payment engine at NPCI, the student information system at a university, or the inventory management system at a Reliance Retail store — is far too large and complex to fit inside a single `main()` function. Module 2 introduces three of the most powerful tools in C programming for managing this complexity: functions, arrays, and strings.

Functions are the building blocks of modular programming. Just as a bank has separate departments for deposits, withdrawals, loans, and customer service — each handling its specific responsibility — a well-designed C program divides its work into functions, each responsible for a clearly defined task. This makes programs easier to write, test, debug, and extend. You will learn how to declare and define functions, pass data into them using value and reference methods, use recursion for elegant solutions to repetitive mathematical problems, and organise return values meaningfully.

Arrays are the workhorses of data storage in C. When a telecom company like Jio needs to track the daily data usage of 490 million subscribers, or when a hospital needs to store temperature readings from 200 patient monitors, they use structured collections of data — arrays. You will learn one-dimensional arrays for sequences, two-dimensional arrays for matrices and tabular data, and how to pass arrays into functions for processing.

Strings — which are simply character arrays in C — enable your programs to handle text. Real-world applications depend heavily on string operations: validating an employee name entered at a biometric terminal, comparing a typed OTP with the stored one, extracting a filename from a full file path, or formatting a transaction receipt. You will master the standard string library functions and learn to write programs that process text reliably.

The module concludes with the critical skill of implementing sorting and searching algorithms — the algorithms that power every product search on Flipkart, every contact lookup on your phone, and every database query across India's digital infrastructure. By the end of Module 2, you will write programs that are modular, efficient, and capable of handling realistic data volumes.

Learning Objectives

By the end of this module, students will be able to:

- Understand the concept of modular programming and explain the structure of function declaration, definition, and calling in C.
- Understand the difference between pass-by-value and pass-by-reference argument passing and describe when each is appropriate.
- Understand the concept of recursion and trace the execution of a recursive function through its base case and recursive steps.
- Apply one-dimensional and two-dimensional arrays to store and process collections of real-world data.
- Apply standard string library functions to perform common text operations and implement basic sorting and searching algorithms on arrays.

Learning Outcomes

Upon successful completion of this module, students will be able to:

- Design and implement modular C programs by breaking a problem into well-defined functions with appropriate parameters and return types.
- Write programs that pass arrays and variables to functions using both pass-by-value and pass-by-reference, and explain the effect on original data.
- Implement recursive solutions for factorial, Fibonacci, and power calculations and trace their execution step by step.
- Create and manipulate 1D and 2D arrays to solve real-world problems such as student marks processing, sales tabulation, and sensor data analysis.
- Apply standard C string functions (strlen, strcpy, strcat, strcmp, strrev) and implement Bubble Sort and Binary Search on arrays of real-world data.

Table of Topics

Sub-Unit	Topic	Coverage
2.1	Introduction to Functions	Modular design, function anatomy, declaration vs definition
2.2	Function Parameters and Return Types	Formal vs actual args, return values, void functions
2.3	Argument Passing: Value vs Reference	Pass-by-value, pass-by-pointer, swap example, side effects
2.4	Recursion	Base case, recursive step, factorial, Fibonacci, tower of Hanoi
2.5	One-Dimensional Arrays	Declaration, initialisation, traversal, real-world data sets
2.6	Arrays and Functions	Passing 1D/2D arrays to functions, array decay to pointer
2.7	Two-Dimensional Arrays	Matrices, row-major storage, marks sheet, sales table
2.8	Sorting Algorithms	Bubble Sort, Selection Sort — logic, trace, complexity
2.9	Searching Algorithms	Linear Search, Binary Search — logic, trace, complexity
2.10	Strings and String Functions	char arrays, gets/puts, strlen, strcpy, strcat, strcmp, strrev, sprintf
2.11	Multidimensional String Arrays	Array of strings, menu lists, city names, product catalogues

2.1 Introduction to Functions

Consider the software powering an SBI ATM. It must handle PIN verification, balance enquiry, cash withdrawal, mini statement, and PIN change — five completely different tasks. If a developer wrote all of this as one giant block of code inside `main()`, the program would become nearly impossible to test, maintain, or update. When a new RBI guideline changes the withdrawal limit, the developer would have to search through thousands of lines of tangled code to find and update the right section.

Functions solve this problem elegantly. A function is a named, self-contained block of code that performs a specific task. Once defined, it can be called (invoked) from anywhere in the program as many times as needed — without rewriting the logic. This principle is called modularity, and it is the foundation of all professional software development.

2.1.1 Anatomy of a C Function

Every function in C has three parts:

- **Function Declaration (Prototype):** Tells the compiler the function's name, return type, and parameter types — before the function is defined. Placed above `main()`.
- **Function Definition:** The actual body of the function — the code that runs when the function is called.
- **Function Call:** The statement inside `main()` (or another function) that triggers the function's execution.

```
/* ATM Module — Function Declaration (Prototype) */

float check_balance(long account_no, char *pin); /* Declaration */

void display_welcome(char *customer_name);      /* Declaration */


/* Function Definition */

void display_welcome(char *customer_name) {

    printf("\nWelcome to SBI ATM, %s!\n", customer_name);

    printf("Your security session starts now.\n");

}


int main() {

    display_welcome("Priya Sharma"); /* Function Call */

    return 0;

}
```

2.1.2 Why Functions?

Benefit	Real-World Impact
Reusability	calculate_gst() written once, called for every transaction — saves rewriting the same 5-line formula 200 times.
Readability	main() becomes a clean sequence of function calls like process_login() → show_menu() → execute_transaction() — easy to follow.

Testing	Each function can be tested independently. If PIN verification is buggy, test <code>verify_pin()</code> in isolation without running the whole ATM program.
Maintainability	When SEBI changes the transaction tax rate, update only the <code>calculate_tax()</code> function — not 50 different places in the code.

2.2 Function Parameters and Return Types

Functions become truly powerful when they can accept input data (parameters) and send back a computed result (return value). Parameters are the function's inputs — the information it needs to do its job. The return type declares what kind of value the function sends back to the caller.

2.2.1 Functions with Parameters and Return Value

Scenario: A telecom billing system needs to calculate the monthly bill for a Jio postpaid customer based on plan type and excess data usage:

```
#include <stdio.h>

/* Function: calculate_postpaid_bill()
 * Returns total bill in rupees based on plan and excess GB used */
float calculate_postpaid_bill(int plan_code, float excess_gb) {
    float base_price, per_gb_rate, total;

    if (plan_code == 1) { base_price = 399.0; per_gb_rate = 20.0; }
    else if (plan_code == 2) { base_price = 599.0; per_gb_rate = 15.0; }
    else { base_price = 999.0; per_gb_rate = 10.0; }
```

```
total = base_price + (excess_gb * per_gb_rate);  
  
return total; /* Send computed value back to caller */  
  
}
```

```
int main() {
```

```
float bill = calculate_postpaid_bill(2, 3.5);
```

```
printf("Jio Postpaid Bill: Rs. %.2f\n", bill);
```

```
/* Output: Jio Postpaid Bill: Rs. 651.50 */
```

```
return 0;
```

```
}
```

2.2.2 void Functions — No Return Value

When a function performs an action but does not compute a result to send back, its return type is void. Examples: printing a receipt, displaying a menu, logging an event.

```
/* Print a formatted transaction receipt — no return value needed */
```

```
void print_receipt(long acc_no, float amount, char *txn_type) {
```

```
printf("==== HDFC BANK TRANSACTION RECEIPT =====\n");
```

```
printf("Account No : %ld\n", acc_no);
```

```
printf("Type      : %s\n", txn_type);
```

```
printf("Amount    : Rs. %.2f\n", amount);
```

```
printf("Status    : SUCCESS\n");
```

```
printf("=====");
```

```
}
```

```
int main() {  
    print_receipt(1234567890L, 5000.0, "NEFT Transfer");  
    return 0;  
}
```

2.2.3 Formal vs Actual Arguments

- Formal Arguments (Parameters): Variable names listed in the function definition — they exist only inside the function. e.g., `plan_code` and `excess_gb` in `calculate_postpaid_bill()`.
- Actual Arguments: The values or variables passed when the function is called. e.g., `2` and `3.5` in `calculate_postpaid_bill(2, 3.5)`.
- The number, order, and type of actual arguments must match the formal parameters. Mismatches cause compilation warnings or runtime errors.

✓ Let's Check

Q.No	Type	Question
1	MCQ	A function declared as <code>void display_menu()</code> can: (A) Return a float value (B) Return an int value (C) Not return any value (D) Only be called once
2	MCQ	Where should a function prototype be placed in a C program? (A) After <code>main()</code> (B) Inside <code>main()</code> (C) Before <code>main()</code> , above all function calls (D) At the end of the file
3	Fill in Blank	The values passed to a function when it is called are called _____ arguments.

4	Fill in Blank	A function with return type _____ does not send any value back to the caller.
5	True/False	A C function can return multiple values using a single return statement. (True/False)
6	True/False	Formal parameters in a function definition act as local variables inside the function. (True/False)

Answers:

Q.No	Answer
1	(C) Not return any value — void means the function performs an action but returns nothing to the caller.
2	(C) Before main(), above all function calls — the prototype informs the compiler about the function's signature before it is called.
3	Actual
4	void
5	False — a return statement can return only one value. To return multiple values, use pointers (pass by reference) or a struct.
6	True — formal parameters are created when the function is called and destroyed when it returns; they behave exactly like local variables.

2.3 Argument Passing: Pass-by-Value and Pass-by-Reference

One of the most important concepts in C functions is understanding what happens to the original variable when you pass it to a function. C supports two mechanisms: pass-by-value and pass-by-reference (via pointers). Choosing the right one determines whether a function can modify the caller's data or only work on a local copy.

2.3.1 Pass-by-Value

In pass-by-value, a copy of the variable's value is sent to the function. The function works on the copy — any changes inside the function have no effect on the original variable in `main()`.

Scenario: A loyalty points system at a BigBazaar store calculates discounted price without changing the original price:

```
#include <stdio.h>

/* This function gets a COPY of price — original is unchanged */

float apply_loyalty_discount(float price, int loyalty_points) {
    if (loyalty_points >= 500)
        price = price * 0.90; /* 10% discount — changes local copy only */
    return price;
}

int main() {
    float original_price = 1200.0;
    int points = 650;
```

```
float discounted = apply_loyalty_discount(original_price, points);
```

```
printf("Original Price : Rs. %.2f\n", original_price);
```

```
/* Still Rs. 1200 — not modified by function */
```

```
printf("Discounted Price: Rs. %.2f\n", discounted);
```

```
/* Output: Rs. 1080.00 */
```

```
return 0;
```

```
}
```

2.3.2 Pass-by-Reference (via Pointers)

In pass-by-reference, the memory address of the variable is passed. The function uses a pointer to directly access and modify the original variable in the caller's memory. Changes made inside the function are reflected in the original variable.

Scenario: A currency exchange counter at an airport needs to swap two account balances (INR ↔ USD) — the original values must change:

```
#include <stdio.h>
```

```
/* Receives ADDRESSES — can modify originals */
```

```
void swap_amounts(float *inr_balance, float *usd_balance) {
```

```
    float temp = *inr_balance;
```

```
    *inr_balance = *usd_balance;
```

```
    *usd_balance = temp;
```

```
}
```

```
int main() {  
  
    float inr = 83500.0; /* INR in customer wallet */  
  
    float usd = 1000.0; /* USD equivalent */  
  
    printf("Before Swap: INR=%.2f, USD=%.2f\n", inr, usd);  
  
    swap_amounts(&inr, &usd); /* Pass addresses */  
  
    printf("After Swap : INR=%.2f, USD=%.2f\n", inr, usd);  
  
    /* INR=1000.00, USD=83500.00 */  
  
    return 0;  
  
}
```

🔍 Analysing Misconceptions: Pass-by-Value Modifies the Original

✗ MYTH: Passing a variable to a function and modifying it inside always changes the original variable.

✓ REALITY: In C, the default is pass-by-value: the function receives a copy. Modifying the copy does not affect the original. To allow a function to change the original, you must explicitly pass the variable's address using & and use a pointer parameter (int *x). This is a fundamental design choice — C deliberately defaults to pass-by-value to prevent accidental modification of caller data.

2.4 Recursion

Recursion is the technique of a function calling itself to solve a problem by breaking it into smaller, identical sub-problems. It is elegant, powerful, and — once understood — very natural for certain classes of problems. Every recursive solution must have two essential components: a base case (when to stop) and a recursive step (how to reduce the problem).

Think of a folder structure on your computer — the Downloads folder contains sub-folders, each of which may contain more sub-folders. To calculate the total size of all files, you use the same "calculate folder size" operation on each sub-folder — recursion in the real world.

2.4.1 Factorial — Billing Penalty Calculation

Scenario: A financial compliance system calculates a compounding penalty. The penalty for n days of delay is proportional to $n!$ (n factorial). $n! = n \times (n-1) \times (n-2) \times \dots \times 1$.

```
#include <stdio.h>

/* Recursive factorial: 5! = 5 × 4 × 3 × 2 × 1 = 120 */
long int factorial(int n) {
    if (n == 0 || n == 1) /* BASE CASE: stop recursion */
        return 1;
    return n * factorial(n - 1); /* RECURSIVE STEP */
}

int main() {
    int delay_days = 5;
```



```
long int penalty_factor = factorial(delay_days);

float base_penalty = 100.0;

printf("Delay: %d days\n", delay_days);

printf("Penalty Factor (5!): %ld\n", penalty_factor);

printf("Total Penalty: Rs. %.2f\n", base_penalty * penalty_factor);

return 0;

}
```

Execution trace for factorial(4):

factorial(4) → 4 × factorial(3) → 4 × 3 × factorial(2) → 4 × 3 × 2 × factorial(1) →
4 × 3 × 2 × 1 = 24

2.4.2 Fibonacci — Stock Price Modelling

Scenario: A quantitative finance intern at Zerodha uses the Fibonacci sequence to identify support and resistance levels in stock price charts — a technique called Fibonacci retracement used widely in technical analysis.

```
/* Fibonacci: F(0)=0, F(1)=1, F(n) = F(n-1) + F(n-2) */
int fibonacci(int n) {
    if (n == 0) return 0;      /* Base case 1 */
    if (n == 1) return 1;      /* Base case 2 */
    return fibonacci(n-1) + fibonacci(n-2); /* Recursive step */
}

int main() {
```

```
int i;

printf("Fibonacci Retracement Levels (first 10):\n");

for (i = 0; i < 10; i++)

    printf("F(%d) = %d\n", i, fibonacci(i));

return 0;

}
```

2.4.3 Power Function — EMI Interest Calculation

Scenario: A bank's loan processing system calculates compound interest using the power function. `power(1.08, 5)` computes $(1.08)^5$ — growth factor for a 5-year FD at 8% per annum.

```
double power(double base, int exp) {

    if (exp == 0) return 1.0;          /* Base case: anything^0 = 1 */

    return base * power(base, exp - 1); /* Recursive step */

}

int main() {

    double principal = 100000.0; /* Rs. 1 lakh */

    double rate = 1.08;          /* 8% annual interest factor */

    int years = 5;

    double maturity = principal * power(rate, years);

    printf("FD Maturity Value after %d years: Rs. %.2f\n", years, maturity);

    /* Output: Rs. 146932.81 */

    return 0;

}
```

```
}
```

💡 Did You Know?

Recursion is used heavily in the file system traversal of Android phones (Linux-based). When you tap "Share All Photos" in Google Photos on your Pixel or Samsung phone, the app recursively walks every sub-directory of your storage using a recursive directory scan — the same concept as the recursive factorial function, but applied to folder trees. The recursive algorithm that does this (called Depth-First Search) was designed by computer scientist Robert Floyd in the 1960s and is still the standard approach today.

2.5 One-Dimensional Arrays

Suppose a hospital in Hyderabad monitors the body temperature of 10 ICU patients every hour. Without arrays, you would need 10 separate variables: temp1, temp2, ..., temp10. Processing all 10 would require 10 separate printf statements, 10 separate if conditions, and would not scale to 100 or 1000 patients. An array solves this by grouping all 10 values under one name, accessed via an index.

An array is a contiguous block of memory holding a fixed number of elements of the same data type. In C, array indices start at 0 — the first element is arr[0], the last is arr[n-1].

2.5.1 Declaration and Initialisation

```
/* ICU Patient Temperature Monitoring — 10 patients */  
float temperature[10];      /* Declaration: 10 float elements */  
  
/* Initialisation at declaration */  
float temperature[10] = {37.2, 38.5, 36.9, 39.1, 37.8,  
                        38.0, 36.5, 40.2, 37.5, 38.8};  
  
/* Partial initialisation — remaining elements set to 0 */  
int daily_steps[7] = {8500, 10200, 7800}; /* [3]–[6] = 0 */  
  
/* Size determined by initialiser list */  
float petrol_price[] = {102.43, 103.12, 101.87, 104.20};  
  
/* Compiler deduces size = 4 */
```

2.5.2 Accessing and Traversing Arrays

Scenario: A fitness app tracks a user's daily step count for a week (7 days) and computes the weekly average:

```
#include <stdio.h>

int main() {

    int steps[7] = {8500, 12300, 9750, 11200, 6800, 14500, 10400};

    int total = 0, i, day_names;

    char *days[] = {"Mon","Tue","Wed","Thu","Fri","Sat","Sun"};

    printf(" Day | Steps | Status\n");

    printf("-----|-----|-----\n");

    for (i = 0; i < 7; i++) {

        total += steps[i];

        char *status = (steps[i] >= 10000) ? "✓ Goal Met" : " Below Goal";

        printf(" %s | %5d | %s\n", days[i], steps[i], status);

    }

    printf("-----|-----|-----\n");

    printf("Weekly Average: %.0f steps/day\n", (float)total / 7);

    return 0;

}
```

2.5.3 Finding Maximum and Minimum

Scenario: A weather station in Bengaluru records hourly temperature for 24 hours. Find the highest and lowest temperatures:

```
#include <stdio.h>

int main() {

    float temp[24] = {
        22.1, 21.8, 21.5, 21.2, 21.0, 21.3, 22.5, 24.8,
        27.2, 29.5, 31.0, 32.4, 33.1, 33.8, 33.5, 32.9,
        31.7, 30.2, 28.6, 27.0, 25.8, 24.5, 23.4, 22.8
    };

    float max_temp = temp[0], min_temp = temp[0];
    int max_hr = 0, min_hr = 0, i;

    for (i = 1; i < 24; i++) {
        if (temp[i] > max_temp) { max_temp = temp[i]; max_hr = i; }
        if (temp[i] < min_temp) { min_temp = temp[i]; min_hr = i; }
    }

    printf("Bengaluru Weather — 24-Hour Report\n");
    printf("Max Temperature: %.1f°C at %02d:00 hrs\n", max_temp, max_hr);
    printf("Min Temperature: %.1f°C at %02d:00 hrs\n", min_temp, min_hr);
    return 0;
}
```

 Let's Check

Q.No	Type	Question
1	MCQ	An array <code>int marks[5]</code> is declared. What is the valid index range? (A) 1 to 5 (B) 0 to 4 (C) 0 to 5 (D) 1 to 4
2	MCQ	What happens if you access <code>arr[10]</code> when the array is declared as <code>int arr[10]</code> ? (A) Returns 0 (B) Compilation error (C) Undefined behaviour — memory outside array (D) Returns the last element
3	Fill in Blank	In C, all elements of an array are stored in _____ memory locations.
4	Fill in Blank	If <code>int scores[5] = {90, 85};</code> is written, the value of <code>scores[4]</code> is _____.
5	True/False	Arrays in C can store elements of different data types in the same array. (True/False)
6	True/False	The size of an array must be a positive integer known at compile time when declared without initialisation. (True/False)

Answers:

Q.No	Answer
1	(B) 0 to 4 — C arrays are zero-indexed. A 5-element array occupies indices 0, 1, 2, 3, 4.

2	(C) Undefined behaviour — C does not perform bounds checking. Accessing <code>arr[10]</code> reads memory beyond the array, which may cause garbage values, crashes, or security vulnerabilities.
3	Contiguous (consecutive)
4	0 — partial initialisation sets unspecified elements to zero.
5	False — all elements of a C array must be of the same data type. For mixed types, use a struct.
6	True — in standard C, variable-length arrays (VLAs, C99+) allow runtime sizes, but basic arrays require a compile-time constant size.

2.6 Arrays and Functions

Passing arrays to functions is a common and essential operation — it allows you to write a single function that processes any array of data, regardless of the specific values. Unlike scalar variables (which are copied in pass-by-value), arrays in C are always passed by reference — the function receives the address of the first element and can directly access and modify the original array.

2.6.1 Passing a 1D Array to a Function

When you write `void process(int arr[], int size)`, the `arr[]` parameter is actually a pointer to the first element. The `size` parameter is required because the function cannot determine the array's length from the pointer alone.

Scenario: A school's exam management system computes the class average and highest score from a student marks array:


```
#include <stdio.h>

/* Compute class average — reads array, does not modify it */
float compute_average(int marks[], int n) {
    int sum = 0, i;
    for (i = 0; i < n; i++) sum += marks[i];
    return (float)sum / n;
}

/* Find highest mark — reads array, does not modify it */
int find_highest(int marks[], int n) {
    int max = marks[0], i;
    for (i = 1; i < n; i++)
        if (marks[i] > max) max = marks[i];
    return max;
}

/* Add grace marks — MODIFIES the original array */
void add_grace_marks(int marks[], int n, int grace) {
    int i;
    for (i = 0; i < n; i++)
        marks[i] += grace;
}
```

```
int main() {  
    int class_marks[] = {78, 85, 62, 91, 74, 88, 55, 96, 70, 83};  
    int n = 10;  
  
    printf("Class Average : %.1f\n", compute_average(class_marks, n));  
    printf("Highest Score : %d\n", find_highest(class_marks, n));  
  
    add_grace_marks(class_marks, n, 3); /* Add 3 grace marks to all */  
  
    printf("After Grace Marks — New Average: %.1f\n",  
        compute_average(class_marks, n));  
  
    return 0;  
}
```

💡 Did You Know?

Arrays and functions together form the backbone of numerical computing. MATLAB, NumPy (Python), and R — the tools used by data scientists at companies like Flipkart and Paytm for recommendation algorithms — are all built on top of C array-processing functions. When a data scientist calls `numpy.mean(sales_data)`, under the hood it calls a C function that iterates over an array pointer, exactly like `compute_average()` above. This is why NumPy is 50–100× faster than pure Python loops.

2.7 Two-Dimensional Arrays

Many real-world data sets are naturally tabular — a matrix of rows and columns. A school's marks sheet has students as rows and subjects as columns. A retail chain's monthly sales report has stores as rows and months as columns. A chessboard, a seating plan for a train, a spreadsheet — all are two-dimensional. C supports 2D arrays to handle exactly this kind of data.

2.7.1 Declaration and Initialisation

```
/* Marks sheet: 5 students × 3 subjects (Maths, Physics, C Lab) */  
  
int marks[5][3] = {  
    {78, 85, 90}, /* Student 1: Rahul */  
    {92, 88, 95}, /* Student 2: Priya */  
    {65, 70, 72}, /* Student 3: Arjun */  
    {88, 91, 87}, /* Student 4: Divya */  
    {74, 68, 80} /* Student 5: Kiran */  
};  
  
/* Access: marks[row][column] — marks[1][2] = Priya's C Lab score = 95 */
```

2.7.2 Processing a 2D Array — Student Result Sheet

Scenario: Generate a complete result sheet for a BCA class — compute totals, percentages, and pass/fail status:

```
#include <stdio.h>  
  
int main() {  
  
    char *names[] = {"Rahul", "Priya", "Arjun", "Divya", "Kiran"};
```

```
char *subjects[] = {"Maths","Physics","C Lab"};

int marks[5][3] = {

    {78,85,90},{92,88,95},{65,70,72},{88,91,87},{74,68,80}

};
```

```
int i, j;
```

```
printf("%-10s | M | Ph | CL | Total | %% | Result\n", "Name");
```

```
printf("-----\n");
```

```
for (i = 0; i < 5; i++) {
```

```
    int total = 0;
```

```
    int pass = 1;
```

```
    for (j = 0; j < 3; j++) {
```

```
        total += marks[i][j];
```

```
        if (marks[i][j] < 40) pass = 0; /* Fail if any subject < 40 */
```

```
    }
```

```
    float percent = (float)total / (3 * 100) * 100;
```

```
    printf("%-10s | %2d | %2d | %2d | %3d |%.1f| %s\n",
```

```
        names[i], marks[i][0], marks[i][1], marks[i][2],
```

```
        total, percent, pass ? "PASS" : "FAIL");
```

```
}
```

```
return 0;
```

```
}
```

2.7.3 Memory Layout of 2D Arrays

In C, a 2D array is stored in row-major order — all elements of row 0 are stored first, then row 1, then row 2, and so on, in a single contiguous block of memory. This means that row-by-row traversal (outer loop over rows, inner loop over columns) is more cache-friendly and faster than column-by-column traversal — an important optimisation consideration in performance-critical applications like image processing and matrix multiplication.

2.8 Sorting Algorithms

Sorting means arranging elements in a defined order — ascending or descending. It is arguably the most studied class of algorithms in computer science, and for good reason: almost every useful data operation benefits from sorted data. Product search results on Amazon India, ranked leaderboards in fantasy cricket apps like Dream11, sorted transaction histories in HDFC NetBanking — all depend on efficient sorting. We study two foundational sorting algorithms here.

2.8.1 Bubble Sort

Bubble Sort works by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. With each full pass through the array, the largest unsorted element "bubbles up" to its correct position at the end — like bubbles rising to the surface of water.

Scenario: Sort customer complaint ticket IDs at Reliance Jio's customer care centre in ascending order for priority processing:

```
#include <stdio.h>

void bubble_sort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp; /* Swap */
            }
        }
    }
}

void print_tickets(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("TKT-%04d ", arr[i]);
    printf("\n");
}

int main() {
```

```
int tickets[] = {4521, 1203, 8867, 3344, 2091, 6750, 1005};

int n = 7;

printf("Before Sort: "); print_tickets(tickets, n);

bubble_sort(tickets, n);

printf("After Sort : "); print_tickets(tickets, n);

return 0;

}
```

Trace (first two passes on array [4521, 1203, 8867, 3344]):

Pass 1: Compare 4521 ↔ 1203 → Swap → [1203, 4521, 8867, 3344]. Compare 4521 ↔ 8867 → No swap. Compare 8867 ↔ 3344 → Swap → [1203, 4521, 3344, 8867].

Time Complexity: $O(n^2)$ in the worst case. Suitable for small arrays ($n < 1000$).

2.8.2 Selection Sort

Selection Sort finds the minimum element in the unsorted portion and places it at the beginning — "selecting" the smallest each time.

```
void selection_sort(float prices[], int n) {
    int i, j, min_idx;
    float temp;

    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++)
            if (prices[j] < prices[min_idx]) min_idx = j;

        /* Swap minimum with first unsorted */
        temp = prices[i];
```

```
        prices[i] = prices[min_idx];

        prices[min_idx] = temp;
    }
}

int main() {

    /* Flipkart product prices — sort ascending for "Price: Low to High" */

    float prices[] = {1299.0, 599.0, 4999.0, 849.0, 2199.0};

    int n = 5;

    selection_sort(prices, n);

    printf("Sorted Prices (Low to High): ");

    for (int i = 0; i < n; i++) printf("Rs.%.0f ", prices[i]);

    printf("\n");

    return 0;

}
```

🔍 Analysing Misconceptions: Bubble Sort and Efficiency

✗ MYTH: Bubble Sort is the best sorting algorithm to use because it is simple to understand.

✓ REALITY: Bubble Sort's $O(n^2)$ complexity makes it practical only for small arrays ($n < 100$). For the 300 million products on Flipkart or Amazon India's catalogue, $O(n^2)$ sorting would take years. Production systems use Timsort (Python's built-in sort), Introsort (C++ STL sort), or Merge Sort — all $O(n \log n)$ algorithms. Understanding Bubble Sort is valuable for learning the concept; always use optimised algorithms for production data.

2.9 Searching Algorithms

Searching means finding whether a specific value exists in a collection and, if so, where it is located. Every meaningful application searches data — validating an account number, looking up a patient record, finding a product by barcode. Two fundamental searching algorithms form the foundation.

2.9.1 Linear Search

Linear Search scans every element from the beginning until the target is found or the array is exhausted. It works on both sorted and unsorted arrays.

Scenario: A pharmacist at a hospital searches for a drug by its batch number in the medicine inventory:

```
#include <stdio.h>

int linear_search(int batch_no[], int n, int target) {
    int i;
    for (i = 0; i < n; i++)
        if (batch_no[i] == target)
            return i; /* Return index where found */
    return -1; /* -1 = not found */
}

int main() {
    int batches[] = {10421, 20835, 30192, 40007, 50348, 60211};
    int n = 6;
    int search = 40007;
```

```
int pos = linear_search(batches, n, search);

if (pos != -1)

    printf("Batch %d found at position %d in inventory.\n", search, pos+1);

else

    printf("Batch %d not found. Check procurement records.\n", search);

return 0;

}
```

Time Complexity: $O(n)$ — in the worst case, checks every element once.

2.9.2 Binary Search — Efficient Lookup on Sorted Data

Binary Search works only on sorted arrays. It repeatedly divides the search space in half — comparing the target with the middle element and discarding the half that cannot contain the target. Like looking up a word in a dictionary: you open to the middle, determine if your word comes before or after, and search only the relevant half.

Scenario: NPCI's fraud detection system looks up a flagged card number from a sorted list of 10 million blocked cards in milliseconds using binary search:

```
#include <stdio.h>

int binary_search(long arr[], int n, long target) {

    int low = 0, high = n - 1, mid;

    while (low <= high) {

        mid = (low + high) / 2;
```

```
if (arr[mid] == target)

    return mid;        /* Found */

else if (arr[mid] < target)

    low = mid + 1;     /* Search right half */
```

```
else

    high = mid - 1;    /* Search left half */
```

```
}
```

```
return -1; /* Not found */
```

```
}
```

```
int main() {
```

```
/* Sorted blocked card numbers */
```

```
long blocked[] = {4111111111111L, 4222222222222L,
```

```
4333333333333L, 4444444444444L,
```

```
4555555555555L, 4666666666666L};
```

```
int n = 6;
```

```
long card = 4333333333333L;
```

```
int result = binary_search(blocked, n, card);
```

```
if (result != -1)
```

```
    printf("ALERT: Card %ld is BLOCKED! Transaction denied.\n", card);
```

```
else
```

```
    printf("Card %ld is clear. Proceed.\n", card);
```

```
return 0;
```

}

Time Complexity: $O(\log n)$ — for 1 million elements, binary search needs only ~20 comparisons. Linear search would need up to 1,000,000.

✓ **Let's Check**

Q.No	Type	Question
1	MCQ	What is the time complexity of Binary Search in the worst case? (A) $O(n)$ (B) $O(n^2)$ (C) $O(\log n)$ (D) $O(1)$
2	MCQ	Binary Search requires the array to be: (A) Filled with unique elements (B) Sorted (C) An integer array (D) Of even size
3	Fill in Blank	Bubble Sort works by repeatedly _____ adjacent elements if they are in the wrong order.
4	Fill in Blank	Linear Search returns _____ when the element is not found in the array.
5	True/False	Binary Search can be applied to an unsorted array and will still find the correct element. (True/False)
6	True/False	Arrays in C are always passed by reference to functions, meaning the function can modify the original array. (True/False)

Answers:

Q.No	Answer
1	(C) $O(\log n)$ — binary search halves the search space with each step; for n elements, it requires at most $\log_2(n)$ comparisons.
2	(B) Sorted — binary search relies on the sorted order to determine which half to discard; it produces incorrect results on unsorted data.
3	Swapping
4	-1 (or any sentinel value — the conventional choice in C is -1)
5	False — binary search requires a sorted array. On unsorted data, it may miss the target or return incorrect results.
6	True — when an array name is passed to a function, C passes the address of the first element. The function operates on the original array and can modify its contents.

2.10 Strings and String Functions

In C, a string is not a separate data type — it is an array of characters terminated by a special null character `'\0'` (ASCII value 0). This null terminator marks the end of the string. Every string function in the C standard library depends on this null terminator to know where the string ends.

Real-world applications constantly manipulate strings: validating a customer's name input at a banking kiosk, comparing an OTP entered by the user with the one sent by the server, formatting a transaction description, parsing a CSV file containing product names. The `<string.h>` header provides a powerful suite of functions for all of these.

2.10.1 String Declaration and Input

```
#include <stdio.h>

#include <string.h>


int main() {

    /* Declaration: must be large enough to hold the string + '\0' */

    char customer_name[50];

    char city[30] = "Bengaluru"; /* Initialised — '\0' added automatically */


    printf("Enter customer name: ");

    /* fgets() is safer than gets() — prevents buffer overflow */

    fgets(customer_name, 50, stdin);


    printf("Welcome, %s", customer_name);

    printf("City: %s\n", city);
```

```

return 0;

}

```

2.10.2 Standard String Functions — <string.h>

Function	Purpose	Real-World Example
strlen(s)	Returns length of string (excluding \0)	Validate OTP length == 6 before submission
strcpy(dest, src)	Copies src into dest	Copy verified customer name into session buffer
strcat(dest, src)	Appends src to end of dest	Build full address: strcat(city, ", Maharashtra")
strcmp(s1, s2)	Compares two strings; 0 = equal	Compare entered password with stored hash string
strcmpi/strcasecmp	Case-insensitive comparison	Accept "yes", "YES", "Yes" all as valid consent
strupr / strlwr	Convert to uppercase / lowercase	Normalise city name input before database lookup
strrev(s)	Reverses the string in-place	Check if a word is a palindrome (Civic, Madam)

strstr(s, sub)	Finds first occurrence of substring	Search for "@" in email address validation
sprintf(buf, fmt, ...)	Format string into a character buffer	Build formatted transaction reference "TXN20240815001"

2.10.3 Practical String Program — OTP Validator

Scenario: Validate an OTP entered by a user for UPI payment authentication:

```
#include <stdio.h>

#include <string.h>

int validate_otp(char *entered, char *actual) {
    if (strlen(entered) != 6) {
        printf("Invalid OTP: Must be exactly 6 digits.\n");
        return 0;
    }
    if (strcmp(entered, actual) == 0)
        return 1; /* OTP matched */
    return 0;    /* OTP mismatch */
}

int main() {
    char server_otp[7] = "482916"; /* Sent by server to mobile */
```



```
char user_input[10];

int attempts = 0;

while (attempts < 3) {

    printf("Enter OTP (attempt %d/3): ", attempts + 1);

    scanf("%s", user_input);

    if (validate_otp(user_input, server_otp)) {

        printf("✓ OTP Verified! Transaction Authorised.\n");

        return 0;

    }

    printf("X Incorrect OTP.\n");

    attempts++;

}

printf("Transaction Blocked: OTP limit exceeded.\n");

return 1;

}
```

💡 Did You Know?

The `strcmp()` function compares strings character by character using their ASCII values. This is exactly how databases like MySQL compare string columns in `WHERE name = "Priya"` queries. MySQL's InnoDB engine, used by millions of Indian startups and enterprises (Zomato, Swiggy, Urban Company), executes these string comparisons in C internally — using the same logic you write when you call `strcmp()`. Understanding C string operations gives you direct insight into how database queries work at the lowest level.

2.11 Multidimensional String Arrays

When a program needs to store a collection of strings — a list of city names, a product catalogue, a menu of options, or a roster of student names — a 2D character array (array of strings) is the appropriate data structure. Each row of the 2D array holds one string.

2.11.1 Declaring and Using a 2D String Array

```
/* Array of 6 city names — each row holds one city */
```

```
char cities[6][20] = {
```

```
    "Mumbai", "Delhi", "Bengaluru",
```

```
    "Hyderabad", "Chennai", "Kolkata"
```

```
};
```

```
/* cities[0] = "Mumbai", cities[2] = "Bengaluru" */
```

```
/* Print all cities with index */
```

```
int i;
```

```
for (i = 0; i < 6; i++)
```

```
    printf("%d. %s\n", i + 1, cities[i]);
```

2.11.2 Application — Railway Station Menu

Scenario: A self-service kiosk at New Delhi Railway Station displays route options. The user enters a city name and the system checks if a direct train is available:

```
#include <stdio.h>

#include <string.h>


int main() {

    char destinations[8][20] = {

        "Mumbai", "Kolkata", "Chennai", "Bengaluru",

        "Hyderabad", "Ahmedabad", "Jaipur", "Lucknow"

    };

    char search_city[20];

    int i, found = 0;


    printf("=== NDLS Route Search ===\n");

    printf("Enter destination city: ");

    scanf("%s", search_city);


    for (i = 0; i < 8; i++) {

        if (strcasecmp(destinations[i], search_city) == 0) {

            printf("✓ Direct trains available to %s from New Delhi.\n",

                destinations[i]);

            found = 1;

            break;

        }

    }

}
```

```
if (!found)

    printf("No direct trains found. Check IRCTC for connecting routes.\n");

return 0;

}
```

🔍 Analysing Misconceptions: String Comparison with ==

✗ MYTH: Two strings in C can be compared using == operator: if (str1 == str2)

✓ REALITY: In C, str1 and str2 are pointers (memory addresses). Using == compares the addresses, not the string contents — it will almost always return false even when the strings have identical text. The correct function is strcmp(str1, str2), which returns 0 when the strings are equal. This is one of the most common C programming bugs, especially for beginners coming from Python or Java where == compares string content directly.

Case Study

■ Case Study: Zepto's Real-Time Inventory and Delivery Dispatch System — Functions, Arrays, and Strings in 10-Minute Grocery Delivery

Background

Zepto, the Bengaluru-based quick-commerce startup (founded 2021), promises 10-minute grocery delivery across 10 Indian cities including Mumbai, Delhi, Hyderabad, Pune, and Chennai. As of 2024, Zepto operates 300+ dark stores (micro-warehouses) and processes over 300,000 orders per day. Each order triggers a real-time inventory check, nearest dark store lookup, and delivery agent dispatch — operations that must complete in under 500 milliseconds. The core dispatch logic, compiled with GCC on Linux servers, is implemented in C and C++.

Module 2 Concepts Applied

- **Functions:** The dispatch system is divided into modular functions — `find_nearest_store()`, `check_stock(store_id, product_id)`, `assign_agent(order_id)`, and `send_notification(customer_phone, message)`. Each function has a single responsibility and is tested independently. When ONDC integration required adding a new payment function in 2023, the team added `process_ondc_payment()` without touching any other module — thanks to modular design.
- **1D Arrays for Inventory:** Each dark store maintains a `float stock[5000]` array for the 5,000 SKUs it carries. When an order arrives, the system calls `check_stock(store_id, sku_id)`, which performs a direct array index lookup: `stock[sku_id]` — $O(1)$ time complexity. This is far faster than a linear search and is the reason Zepto can check 50 items in a single cart in microseconds.
- **2D Arrays for Zone-Store Mapping:** A 2D array `int delivery_zones[300][20]` maps each of the 300 dark stores to its 20 serviceable pin codes. When an

order arrives from pin code 560034, the system scans this 2D array to find which dark store services that zone — using a function `find_store_for_pincode(int zones[][], int pin)`.

- **Sorting and Searching:** Available delivery agents are stored in a float `agent_distance[50]` array sorted by distance from the dark store (insertion sort, $O(n^2)$ but n is small at ≤ 50). Binary search is used for product catalogue lookups — a sorted int `product_ids[5000]` array allows $O(\log n)$ product validation for each cart item.
- **Strings:** Order confirmation messages are built using `sprintf()`: `sprintf(msg, "Order #%ld confirmed! Delivery by %s. Track: zepto.in/track/%ld", order_id, eta_string, order_id)`. The 300-character message is then passed to the SMS gateway API. `strcmp()` validates customer city input before routing to the correct regional server cluster.

Outcome

- Zepto achieved 92% of deliveries completed within 10 minutes in FY2023–24 (as per Zepto investor disclosure). The C-based dispatch engine completes the full order-to-dispatch pipeline (inventory check, agent assignment, notification) in 180–250 ms on average.
- The modular function architecture allowed Zepto's 200-engineer team to independently develop, test, and deploy the payment module, inventory module, and dispatch module — reducing release cycle time by 40% compared to their initial monolithic prototype.

Reference: Zepto Official Blog (blog.zepto.com), 2024; NASSCOM Indian Startup Report 2024; YourStory — Zepto Tech Architecture Deep Dive, March 2024.

Summary

Module 2 has significantly expanded your C programming capability — from writing simple sequential programs to building modular, data-driven solutions that mirror real-world software systems.

Functions:

- Functions divide programs into manageable, reusable modules. A function has a declaration (prototype), definition (body), and call site. void functions perform actions; typed functions return computed values.
- Formal parameters receive copies of actual argument values (pass-by-value). To allow a function to modify the original variable, pass its address using & and use a pointer parameter (pass-by-reference).
- Recursion enables elegant solutions for self-similar problems: factorial, Fibonacci, power. Every recursive function needs a base case (termination) and a recursive step (problem reduction).

Arrays:

- A 1D array is a contiguous block of same-type elements accessed by index (0 to n-1). Partial initialisation fills remaining elements with zero.
- Arrays are always passed to functions by reference — the function receives the first element's address and can directly modify the original data. The size must always be passed separately.
- 2D arrays store tabular data (marks sheets, sales tables) in row-major order. Access via `arr[row][col]`. Row-major traversal is cache-efficient.

Sorting and Searching:

- Bubble Sort ($O(n^2)$): compares and swaps adjacent elements repeatedly. Selection Sort ($O(n^2)$): selects the minimum and places it at the front. Both are pedagogically important but practically limited to small n .
- Linear Search ($O(n)$): scans all elements; works on unsorted arrays. Binary Search ($O(\log n)$): halves the search space each step; requires sorted array. For $n = 1$ million, binary search takes ~ 20 steps vs 1,000,000 for linear.

Strings:

- Strings in C are null-terminated char arrays. Key library functions: `strlen` (length), `strcpy` (copy), `strcat` (concatenate), `strcmp` (compare — returns 0 for equal), `strrev` (reverse), `sprintf` (formatted string building).
- Never compare strings with `==`; always use `strcmp()`. A 2D char array (array of strings) stores multiple strings — each row is one string, useful for city lists, product names, menu options.

Glossary

Term	Definition
Function	A named, self-contained block of code that performs a specific task. Can accept parameters and return a value.
Prototype (Declaration)	A forward declaration of a function that tells the compiler its name, return type, and parameter types before the function definition appears.
Pass-by-Value	A parameter-passing method where a copy of the variable's value is passed to the function. Changes inside the function do not affect the original.
Pass-by-Reference	Passing the memory address of a variable to a function (using &). The function uses a pointer to directly access and modify the original variable.
Recursion	A programming technique where a function calls itself to solve a problem by reducing it to smaller instances. Requires a base case to terminate.
Array	A contiguous block of memory storing a fixed number of elements of the same data type, accessed via a zero-based integer index.
Bubble Sort	A sorting algorithm that repeatedly compares and swaps adjacent elements if out of order. Time complexity $O(n^2)$. Suitable for small datasets.
Binary Search	A search algorithm that repeatedly halves the search space by comparing the target with the middle element. Requires a sorted array. $O(\log n)$ complexity.
String	In C, a string is a null-terminated character array (<code>char[]</code>). The null character <code>'\0'</code> marks the end of the string and is required by all string functions.
strcmp()	Standard C library function that compares two strings lexicographically. Returns 0 if equal, negative if $s1 < s2$, positive if $s1 > s2$.

Self Assessment Questions

Section A: Multiple Choice Questions (MCQ)

Choose the most appropriate answer for each question.

1. What is the output of: `int x = 5; void double_it(int n) { n = n * 2; } double_it(x); printf("%d", x);`

- A) 10
- B) 5
- C) 0
- D) Compilation error

2. Which of the following function signatures allows a function to modify the original variable passed to it?

- A) `void modify(int n)`
- B) `void modify(int *n)`
- C) `int modify(int n)`
- D) `float modify(float n)`

3. What is the base case in the recursive factorial function?

- A) `return n * factorial(n-1)`
- B) `if (n > 0) return 1`
- C) `if (n == 0 || n == 1) return 1`
- D) `if (n < 0) return -1`

4. An array is declared as `float salary[8]`. What is the index of the last element?

- A) 8
- B) 7
- C) 6
- D) 9

5. When an array is passed to a function in C, what is actually passed?
- A) A copy of all elements
 - B) The size of the array
 - C) The address of the first element
 - D) The value of the last element
6. In a 2D array `int table[3][4]`, how many total elements does it hold?
- A) 7
 - B) 3
 - C) 4
 - D) 12
7. What is the time complexity of Bubble Sort in the worst case?
- A) $O(\log n)$
 - B) $O(n)$
 - C) $O(n \log n)$
 - D) $O(n^2)$
8. Binary Search returns -1 when:
- A) The array is empty
 - B) The array is not sorted
 - C) The target element is not found
 - D) The array has duplicate values
9. Which function returns the number of characters in a string (excluding the null terminator)?
- A) `strcount()`
 - B) `strlen()`
 - C) `strsize()`
 - D) `strcap()`
-

10. What does `strcmp("apple", "apple")` return?
- A) 1
 - B) -1
 - C) 0
 - D) "true"
11. Which header file must be included to use string functions like `strlen` and `strcmp`?
- A) `#include <stdio.h>`
 - B) `#include <stdlib.h>`
 - C) `#include <string.h>`
 - D) `#include <ctype.h>`
12. What is the output of: `char city[30] = "Mumbai"; printf("%lu", strlen(city));`
- A) 30
 - B) 7
 - C) 6
 - D) 31
13. `int arr[5] = {10, 20, 30};` What is the value of `arr[4]`?
- A) 30
 - B) Garbage value
 - C) 0
 - D) Compilation error
14. Which argument-passing technique is required to swap two variables using a function in C?
- A) Pass-by-value
 - B) Pass-by-reference (using pointers)
 - C) Pass-by-name

D) Pass-by-return

15. What is the maximum number of values a C function can return using the return statement?

A) 2

B) 5

C) 1

D) Unlimited

16. Which sorting algorithm finds the minimum element and places it at the beginning of the unsorted portion?

A) Bubble Sort

B) Merge Sort

C) Selection Sort

D) Quick Sort

17. A recursive function without a proper base case will cause:

A) Slow execution

B) Infinite recursion and stack overflow

C) A compilation error

D) Return 0 automatically

18. strcat(dest, src) does which of the following?

A) Copies src into dest

B) Compares dest and src

C) Appends src to the end of dest

D) Returns the length of dest + src

19. In Binary Search, if the target is greater than the middle element, the next search is in:

A) The left half

B) The entire array again

- C) The right half
- D) A new sorted array

20. Zepto's inventory stores 5,000 products in float stock[5000]. Accessing stock[sku_id] directly has time complexity:

- A) $O(n)$
- B) $O(\log n)$
- C) $O(n^2)$
- D) $O(1)$

Section B: True / False

State whether each statement is TRUE or FALSE.

1. In C, a function must always return a value — functions with void return type are not valid.
2. Recursion in C always uses more memory than an equivalent iterative (loop-based) solution, due to the stack frames created for each recursive call.
3. When a 1D array is passed to a function, the function can determine the array's size using sizeof() inside the function.
4. Binary Search is always faster than Linear Search, regardless of the array size or whether the array is sorted.
5. Comparing two C strings using == compares their memory addresses, not their character content.

Section C: Short Answer Questions

Answer each question with a short program or explanation (3–5 sentences / code).

1. Write a C function `int count_vowels(char str[])` that accepts a string (like a customer name or city name) and returns the number of vowels (a, e, i, o, u — case-insensitive) in it. Call it from `main()` and print the result.
2. Write a modular C program with two functions: one to accept marks of 5 students into a 1D array, and another to compute and return the class average. Use proper function prototypes and call both from `main()`.
3. Write a C program using Bubble Sort to sort the following petrol prices in ascending order: {102.50, 98.40, 105.10, 99.80, 103.30}. Print the array before and after sorting.
4. Explain the difference between pass-by-value and pass-by-reference in C with a real-world banking example. When would you use each?
5. Write a recursive C function `int sum_digits(int n)` that returns the sum of digits of a number. For example: `sum_digits(1234)` should return 10 (1+2+3+4). Real-world context: calculate the Luhn checksum digit used in credit card validation.

Self Assessment: Answer Keys

MCQ

Q.No	Correct Answer & Explanation
1	B) 5 — n is a local copy of x (pass-by-value). Doubling n inside the function does not affect x. After the call, x still holds 5.
2	B) void modify(int *n) — using a pointer parameter allows the function to access and modify the original variable via *n = ...
3	C) if (n == 0 n == 1) return 1 — these are the base cases that stop the recursion. Without them, the function would recurse infinitely.
4	B) 7 — a float salary[8] array has 8 elements at indices 0 through 7. Index 8 is out of bounds.
5	C) The address of the first element — array names in C decay to a pointer to their first element when passed to a function.
6	D) 12 — a 3×4 array has 3 rows × 4 columns = 12 total elements.
7	D) $O(n^2)$ — Bubble Sort makes n-1 passes, each scanning up to n-1 elements. Total comparisons $\approx n^2/2 \rightarrow O(n^2)$.
8	C) The target element is not found — -1 is the conventional sentinel return value for "not found" in search functions.
9	B) strlen() — from <string.h>. Returns the count of characters up to (but not including) the null terminator.
10	C) 0 — strcmp returns 0 when both strings are exactly equal character by character.
11	C) #include <string.h> — all string manipulation functions (strlen, strcmp, strcpy, strcat, etc.) are declared in <string.h>.

12	C) 6 — "Mumbai" has 6 characters (M-u-m-b-a-i). <code>strlen()</code> counts characters, not the null terminator. The array size 30 is irrelevant.
13	C) 0 — partial initialisation in C sets all remaining elements to 0. <code>arr[3]</code> and <code>arr[4]</code> are both 0.
14	B) Pass-by-reference (using pointers) — to swap two variables, the function must modify the originals. This requires passing their addresses.
15	C) 1 — a C return statement can return exactly one value. Use pointers or structs to "return" multiple values.
16	C) Selection Sort — finds the minimum in the unsorted portion and places it at the start with one swap per pass.
17	B) Infinite recursion and stack overflow — without a base case, each call creates a new stack frame until the call stack is exhausted, crashing the program with a "Segmentation fault" or stack overflow error.
18	C) Appends <code>src</code> to the end of <code>dest</code> — <code>strcat</code> concatenates; <code>strcpy</code> copies; <code>strcmp</code> compares.
19	C) The right half — if <code>target > arr[mid]</code> , the target must be in <code>arr[mid+1..high]</code> . Set <code>low = mid + 1</code> .
20	D) $O(1)$ — direct array index access (<code>stock[sku_id]</code>) is constant time — the CPU computes the memory address as <code>base_address + (sku_id × sizeof(float))</code> in one step, regardless of array size.

True / False Answer Key

Q.No	Answer / Explanation
1	FALSE — void is a completely valid return type in C. Functions declared as void perform actions (printing, updating, logging) without returning a computed value. void functions are common and essential in well-structured programs.
2	TRUE — each recursive call creates a new stack frame (storing local variables, return address, and parameters). An iterative equivalent reuses the same frame. For deep recursion (e.g., factorial(100000)), this can exhaust the stack. This is a real trade-off between elegance and memory efficiency.
3	FALSE — when passed to a function, an array decays to a pointer. sizeof() inside the function returns the size of the pointer (typically 8 bytes on 64-bit), NOT the full array size. This is why the array size must always be passed as a separate parameter (e.g., void process(int arr[], int n)).
4	FALSE — binary search requires a sorted array and is $O(\log n)$. On an unsorted array, binary search may miss elements. Linear search is $O(n)$ but works on any array. For very small arrays ($n < 10$), linear search may be faster due to lower overhead. Binary search is superior only when the array is sorted and n is large.
5	TRUE — in C, a string variable (char *str or char str[]) holds a memory address. The == operator compares these addresses. Two different string variables with identical content have different addresses → == returns false. Always use strcmp() for content comparison.

Short Answer Key

Q.No	Model Answer
1	<pre>int count_vowels(char str[]) { int i, count = 0; char vowels[] = "aeiouAEIOU"; for (i = 0; str[i] != '\0'; i++) if (strchr(vowels, str[i])) count++; return count; } int main() { char name[50] = "Priya Menon"; printf("Vowels in '%s': %d\n", name, count_vowels(name)); /* Output: Vowels in 'Priya Menon': 4 */ return 0; }</pre>
2	<pre>float compute_average(int marks[], int n); void input_marks(int marks[], int n); void input_marks(int marks[], int n) { int i; for (i = 0; i < n; i++) { printf("Marks of student %d: ", i+1); scanf("%d", &marks[i]); } } float compute_average(int marks[], int n) { int sum = 0, i; for (i = 0; i < n; i++) sum += marks[i]; return (float)sum / n; } int main() { int marks[5]; input_marks(marks, 5); printf("Class Average: %.2f\n", compute_average(marks, 5)); return 0; }</pre>
3	<pre>void bubble_sort(float a[], int n) { int i, j; float t; for (i = 0; i < n-1; i++) for (j = 0; j < n-1-i; j++) if (a[j] > a[j+1]) { t=a[j]; a[j]=a[j+1]; a[j+1]=t; } } int main() { float p[] = {102.50,98.40,105.10,99.80,103.30}; int n=5, i; printf("Before: "); for(i=0;i<n;i++) printf("%.2f ",p[i]); bubble_sort(p,n); printf("\nAfter : "); for(i=0;i<n;i++) printf("%.2f ",p[i]); /* After: 98.40 99.80 102.50 103.30 105.10 */ return 0; }</pre>
4	Pass-by-value: A copy of the variable is passed. The function cannot modify the original. Example: computing GST — float gst = calc_gst(price) — price is unchanged after the call; only the returned gst value is used. Use when you want the function to use a value without changing it. Pass-by-reference: The variable's memory address is passed via &. The function modifies the original through the pointer. Example: update_balance(&balance, debit_amount) — balance in main() is directly

	reduced. Use when the function must change the caller's data, like after a successful ATM withdrawal.
5	<pre>int sum_digits(int n) { if (n == 0) return 0; /* Base case */return (n % 10) + sum_digits(n / 10); /* Last digit + sum of rest */ } int main() { int card_last4 = 1234; printf("Digit sum: %d\n", sum_digits(card_last4)); /* Output: 10 (1+2+3+4) — used in Luhn algorithm for credit card validation */ return 0; }</pre>

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA

